University of Central Punjab

Faculty of Information Technology & Computer Science
Department of Applied Computing & Technologies

Agent Based Modeling

Assignment 1 – Smart Food Delivery Agent

Seven single-agent architectures compared: simple reflex, model-based reflex, goal-based, utility-based, learning, hierarchical and contextual

Course name	Agent Based Modeling
Assignment no.	1 (Agent Types and Architectures)
Section	AG2
Instructor's full name	Zeshan Hussain Chohan
Group members	Syed Raza Abbas Zaidi L1F23BSAI0011 Sher Ali Saleem L1F23BSAI0017 Syed Khesham Ali Raza L1F23BSAI0022
Due date	18 October 2026

Deliverables in this document: code for all agents, one architecture diagram per agent, outputs of the three test cases, a comparison and a conclusion.

Overview

The Contextual Agent was the most suitable of the seven: it delivered on time in all three required tests and, on 1,000 unseen random deliveries, it was on time 96.5% of the time against 96.8% for a hindsight oracle that always knows the truly fastest route.

All seven agent types in the assignment table were built, because the brief says “six” in places but lists seven (it adds the Contextual Agent). Every agent faces the same simulated delivery problem, and each is judged by what actually happens when its chosen route is ridden.

- **Code:** one Python file per agent, a shared simulator, a test runner and a benchmark (full listings below).
- **Diagrams:** one architecture diagram per agent: Input/Perception → Decision Process → Action → Environment, plus the component that defines that agent.
- **Results:** the three required situations (normal; heavy traffic with several routes; heavy traffic + rain + urgent), then a 1,000-delivery benchmark.
- **Comparison and conclusion:** how the agents differ and which one fits the scenario best.

Limitation: the simulator was designed for this assignment and includes effects that only some agents can discover (for example an underpass that floods in rain). The results show how each architecture uses information; they are not evidence about real-world delivery performance.

The agent taxonomy follows Russell and Norvig, *Artificial Intelligence: A Modern Approach*, Chapter 2.

The simulated world

Each delivery is one decision: which route to ride. The simulator keeps two views apart: what an agent can **sense** (traffic readings, weather, distances, deadline and context tags) and what is **really true** (used only afterwards to judge the decision).

Route network (the same in every test)

Route	Distance (km)	Toll (Rs)	Note
A	3.5	0	shortest route
B	5.5	0	passes an underpass
C	4.5	60	toll road

Published model (what the live feed lets an agent predict)

- Speed by traffic: light 40 km/h, normal 30 km/h, heavy 12 km/h; rain multiplies speed by 0.8.
- Delivery time = 7 min fixed (5 pick-up + 2 hand-over) + distance ÷ speed.
- Cost = Rs 8 per km + toll.

Real effects the feed does not show (an agent can only find them out from history or from context)

- Route A in heavy traffic takes 1.30× the predicted travel time (market spill-over).
- Route B in rain takes 1.40× (the underpass floods).
- The toll road C takes 0.95× the prediction.
- Rush hour adds 10% on every route that is not light.
- Live alerts add minutes: accident +12, road works +8, procession +15.
- Random noise (5%) is added only to the past deliveries used for training, never to the three tests.

The three required test situations

Test	Weather	Urgent, deadline	Traffic A / B / C	Extra conditions
1. Normal traffic, good weather	clear	no, 35 min	normal / normal / normal	none
2. Heavy traffic, several routes	clear	no, 30 min	heavy / light / normal	accident alert on C; the live traffic feed for B is down when the agent decides
3. Heavy traffic, rain, urgent	rain	yes, 20 min	heavy / light / normal	rush hour, VIP customer, hot food

What really happens on each route (true delivery time, minutes)

Test	Route A	Route B	Route C	Deadline
1	14.0	18.0	15.5	35
2	29.8	15.2	27.5	30
3	38.3 **(late)**	21.4 **(late)**	18.8	20

In Test 3 only route C arrives on time, so it is the only correct answer. In Test 2 every route is on time, but route B is roughly twice as fast as A or C.

Each agent runs the same way: it perceives when the order is accepted, perceives again at pick-up (decision time), chooses a route, and the simulator then rides that route and reports the true time, whether it was on time, and the cost.

Agent 1 – Simple Reflex Agent

Behaviour: it looks only at the current percept and fires the first matching IF-THEN rule. It has no memory, no deadline reasoning and no cost reasoning, so it cannot tell whether the route it picks will really arrive in time.

Rules: R1 IF the shortest route is heavy THEN take the lightest-traffic alternative. R2 IF it rains, the shortest route is normal and a light route exists THEN take the light route. R3 otherwise take the shortest route.

In the tests: route A in Test 1; route C in Test 2 (the live feed for B was down, so it could not see the light route, and it paid the toll for a 27.5 min ride); route B in Test 3 (late by 1.4 min).

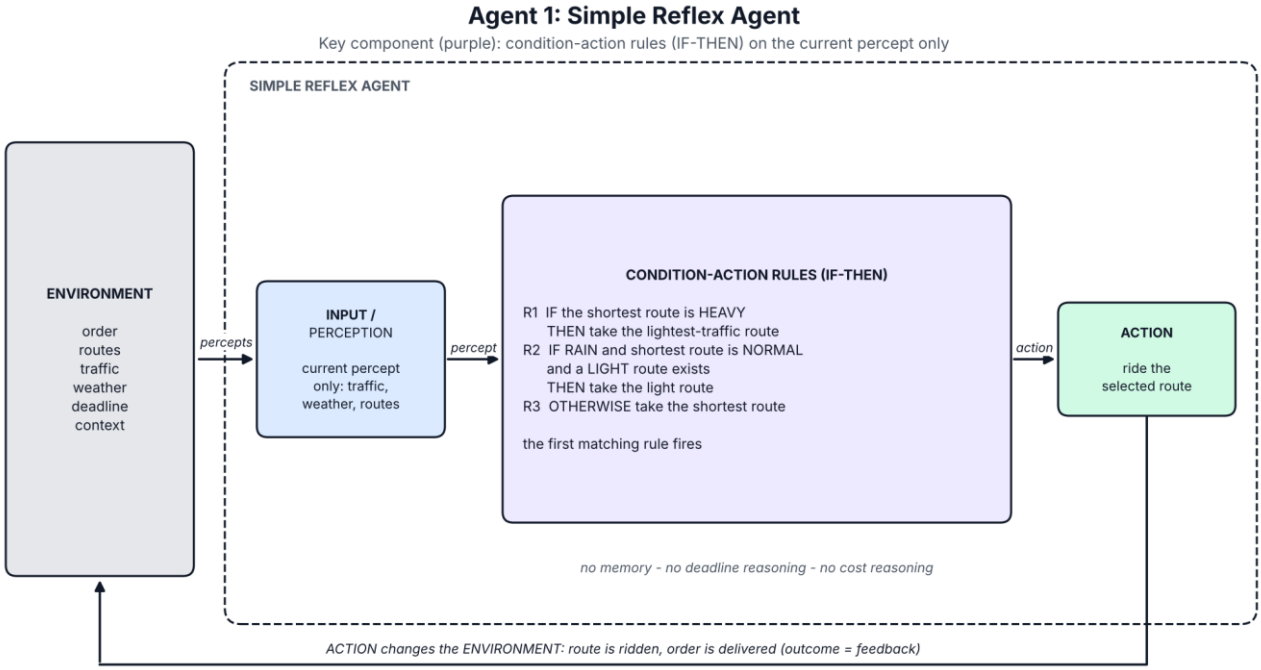


Figure 1. Architecture of Simple Reflex Agent.

Code: `agent1_simple_reflex.py`

```
"""Agent 1 - Simple Reflex Agent.
Decides ONLY from the current percept with fixed IF-THEN (condition-action) rules.
No memory, no deadline reasoning, no cost reasoning."""
from base import Agent, Decision
from environment import TRAFFIC_RANK


def shortest(p):
    return min(p.routes, key=lambda r: r.distance_km)


def lightest_alternative(p, avoid):
    """Lightest traffic among the OTHER routes whose live reading is available right now."""
    known = [r for r in p.routes if r.name != avoid.name and r.traffic is not None]
    return min(known, key=lambda r: (TRAFFIC_RANK[r.traffic], r.distance_km)) if known else avoid


# (rule text, condition(percept), action(percept) -> route); the first matching rule fires
RULES = [
    ("R1: IF shortest route is HEAVY THEN take the lightest-traffic alternative",
     lambda p: shortest(p).traffic == "heavy",
     lambda p: lightest_alternative(p, shortest(p))),
    ("R2: IF RAIN and shortest route is NORMAL and a LIGHT route exists THEN take the light route",
     lambda p: p.weather == "rain" and shortest(p).traffic == "normal"
               and any(r.traffic == "light" for r in p.routes),
     lambda p: min((r for r in p.routes if r.traffic == "light"), key=lambda r: r.distance_km)),
    ("R3: OTHERWISE take the shortest route",
     lambda p: True,
     shortest),
]


class SimpleReflexAgent(Agent):
    name = "1. Simple Reflex"

    def decide(self, p):
        for text, condition, action in RULES:
            if condition(p):
                return Decision(action(p).name, text)
```

Agent 2 – Model-Based Reflex Agent

Behaviour: the rules fire on an internal state (current route, last known traffic of every route, weather, order status) that is updated by every percept and by the agent's own action. When a live traffic reading is missing it falls back to the remembered value instead of ignoring the route.

In the tests: Test 2 shows the difference from Agent 1: the live feed for B was down, but the state still remembered B as light, so it chose B (15.2 min). In Test 3 it chose B again, which floods in rain (21.4 min, late).

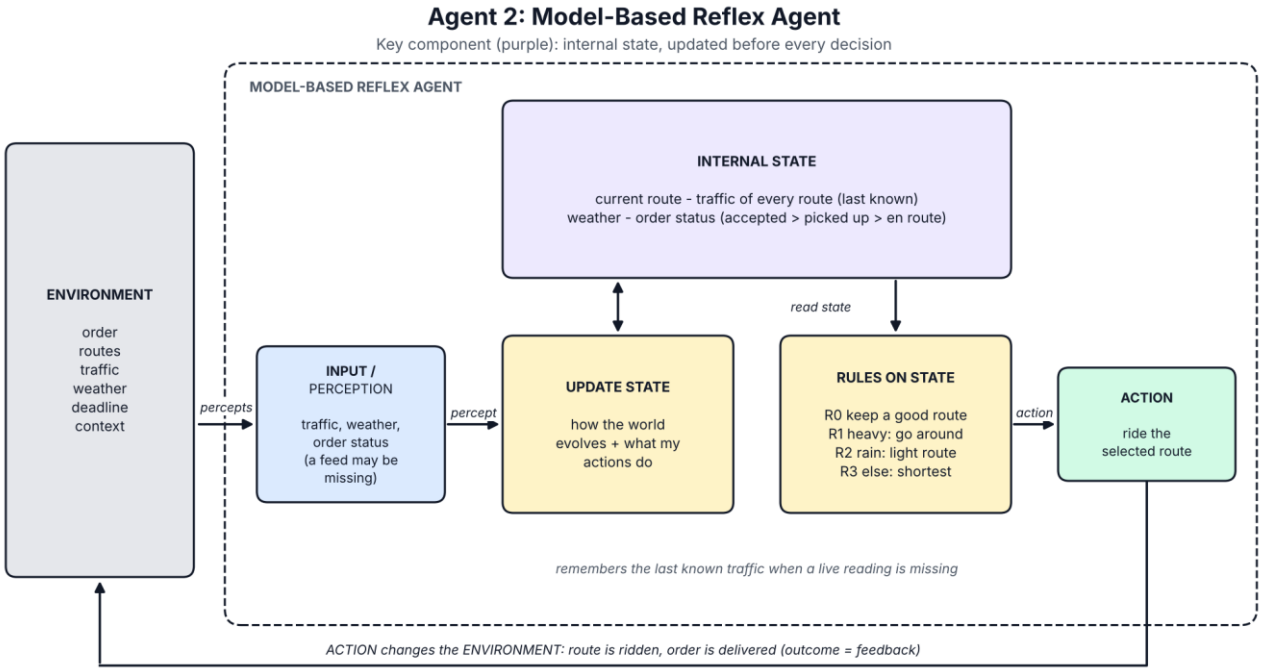


Figure 2. Architecture of Model-Based Reflex Agent.

Code: `agent2_model_based.py`

the shared `StatefulAgent` base class is listed under Shared code

```
"""Agent 2 - Model-Based Reflex Agent.
Same kind of IF-THEN rules, but they fire on an INTERNAL STATE (current route, traffic per route,
weather, order status) that is updated from every percept - so a missing live reading is
replaced by the last known value instead of being ignored."""
from base import StatefulAgent, Decision
from environment import TRAFFIC_RANK


class ModelBasedReflexAgent(StatefulAgent):
    name = "2. Model-Based Reflex"

    def decide(self, p):
        st = self.state                                          # 1) state was updated by
observe()
        traffic = {r.name: self.traffic_of(r) for r in p.routes}  # 2) rules read the STATE, not the
raw percept
        remembered = [r.name for r in p.routes if r.traffic is None]
        short = min(p.routes, key=lambda r: r.distance_km)
        lights = [r for r in p.routes if traffic[r.name] == "light"]

        if st["current_route"] and traffic[st["current_route"]] != "heavy":
            route, why = st["current_route"], "R0: already on a route that is not heavy -> keep it"
        elif traffic[short.name] == "heavy":
            alts = [r for r in p.routes if r.name != short.name]
            route = min(alts, key=lambda r: (TRAFFIC_RANK[traffic[r.name]], r.distance_km)).name
            why = f"R1: state says {short.name} is heavy -> lightest alternative"
        elif st["weather"] == "rain" and traffic[short.name] == "normal" and lights:
            route, why = min(lights, key=lambda r: r.distance_km).name, "R2: rain + normal traffic -
> light route"
        else:
            route, why = short.name, "R3: otherwise shortest route"

        if remembered:
            why += f" (used REMEMBERED traffic for {', '.join(remembered)}: live feed down)"
        self.commit(route)                                       # 3) action updates the state
        return Decision(route, why + f" | state: status={st['order_status']},
weather={st['weather']}, traffic={traffic}")
```

Agent 3 – Goal-Based Agent

Behaviour: the goal is to deliver within the required time. For each route, shortest first, it predicts the delivery time from its state and runs a goal test (predicted time ≤ deadline); the first route that passes is taken. If none passes, it takes the fastest and flags the goal as “at risk”. The test is yes/no, so it stops at a route that merely scrapes through.

In the tests: route A in Test 1. In Test 2 it took the heavy-traffic route A because 24.5 min predicted is within 30 min; the real time was 29.8 min, only 0.2 min inside the deadline, while B needed 15.2 min. In Test 3 route A failed the test (28.9 min predicted) and C passed (18.2 min), so it took C and was on time (18.8 min).

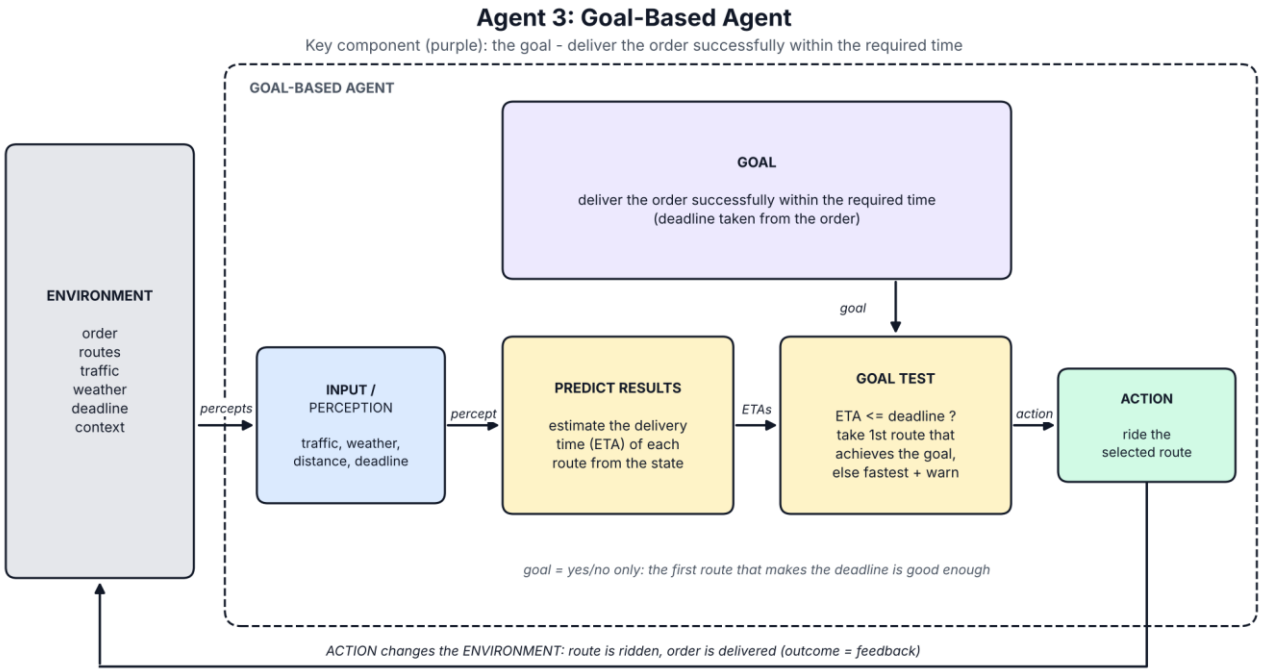


Figure 3. Architecture of Goal-Based Agent.

Code: `agent3_goal_based.py`

```
"""Agent 3 - Goal-Based Agent.
GOAL: deliver the order successfully within the required time.
For each candidate route it PREDICTS the delivery time and TESTS it against the goal.
It is a satisficer: the first route that achieves the goal is good enough (cost/comfort are
ignored)."""
from base import StatefulAgent, Decision
from environment import nominal_eta


class GoalBasedAgent(StatefulAgent):
    name = "3. Goal-Based"

    def decide(self, p):
        tried = []
        for r in sorted(p.routes, key=lambda r: r.distance_km):          # simplest (shortest) route
first
            eta = nominal_eta(r.distance_km, self.traffic_of(r), p.weather)   # predict the result
            tried.append((r.name, eta))
            if eta <= p.deadline_min:                                    # goal test
                self.commit(r.name)
                return Decision(r.name, f"Goal test passed: predicted {eta:.1f} min <=
{p.deadline_min} min "
                                        f"(checked {', '.join(f'{n}={e:.1f}' for n, e in tried)})")
        name, eta = min(tried, key=lambda t: t[1])                       # no route achieves the
goal
        self.commit(name)
        return Decision(name, f"No route meets {p.deadline_min} min; fastest predicted is {eta:.1f}
min",
                        flags=["goal at risk"])
```

Agent 4 – Utility-Based Agent

Behaviour: it predicts time, distance, traffic and cost for every route, converts them into one utility number U(route) with weights (time 0.50, distance 0.15, traffic 0.15, cost 0.20) plus a heavy penalty for arriving after the deadline, and takes the route with the highest U. Unlike the goal test, utility says how good a route is, not only whether it passes.

In the tests: route A in Test 1 (U = -0.38 against -0.50 for B and -0.56 for C). Test 2: route B (U = -0.42 against -0.60 for C and -0.67 for A), 15.2 min at Rs 44. Test 3: route B again (U = -0.60 against -0.79 for C): it trusts the traffic feed, cannot know the underpass floods, and arrives in 21.4 min (late).

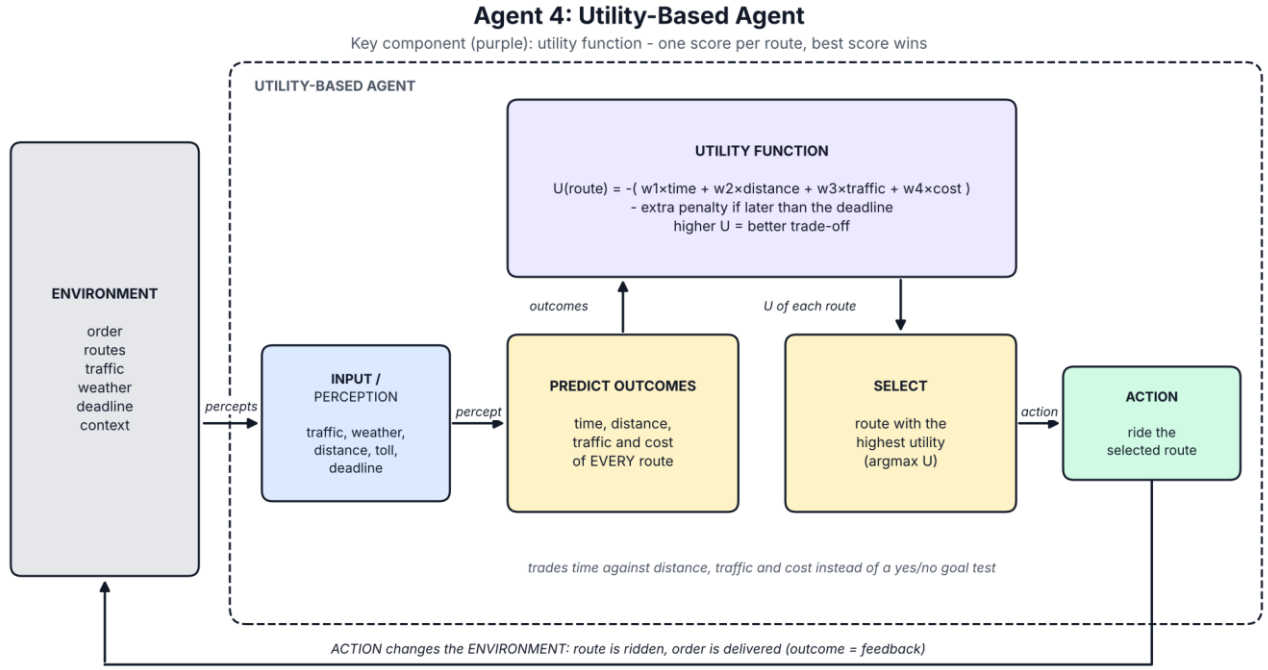


Figure 4. Architecture of Utility-Based Agent.

Utility function (in `base.py`)

```
DEFAULT_WEIGHTS = {"time": 0.50, "distance": 0.15, "traffic": 0.15, "cost": 0.20}
LATE_PENALTY = 3.0

def utility(eta, distance_km, traffic, cost, deadline, w=DEFAULT_WEIGHTS,
late_penalty=LATE_PENALTY):
    """Higher is better. Every factor is scaled to about 0..1, weighted, and subtracted;
    arriving after the deadline adds a large extra penalty that grows with the lateness."""
    loss = (w["time"] * eta / deadline + w["distance"] * distance_km / 10
            + w["traffic"] * TRAFFIC_SCORE[traffic] + w["cost"] * cost / 100)
    if eta > deadline:
        loss += late_penalty * (1 + (eta - deadline) / deadline)
    return -loss
```

Code: `agent4_utility_based.py`

```
"""Agent 4 - Utility-Based Agent.
Compares ALL routes on delivery time, distance, traffic and cost, turns them into ONE utility
number per route, and selects the route with the best (highest) utility."""
from base import StatefulAgent, Decision, utility
from environment import nominal_eta, route_cost


class UtilityBasedAgent(StatefulAgent):
    name = "4. Utility-Based"

    def score(self, p, r):
        traffic = self.traffic_of(r)
        eta = nominal_eta(r.distance_km, traffic, p.weather)
        cost = route_cost(r.distance_km, r.toll)
        return utility(eta, r.distance_km, traffic, cost, p.deadline_min), eta, cost

    def decide(self, p):
        scores = {r.name: self.score(p, r) for r in p.routes}
        best = max(scores, key=lambda n: scores[n][0])
        table = ", ".join(f"{n}: U={scores[n][0]:.2f} ({scores[n][1]:.0f} min, Rs
{scores[n][2]:.0f})"
                          for n in sorted(scores))
        self.commit(best)
        return Decision(best, f"Highest utility -> {best} | {table}")
```

Agent 5 – Learning Agent

Behaviour: it has the four classic parts. The performance element picks the best-utility route using learned delay factors; the critic compares the real delivery time with the expected one; the learning element updates the factor for that route, traffic level and weather (a running average, starting from 1.0 = trust the published model); the problem generator sometimes tries another route during training. It was trained on 600 simulated past deliveries (with 5% random noise), then frozen for the tests.

In the tests: before training it behaves exactly like Agent 4. After training only Test 3 changes: it moves from route B (21.4 min, late) to route C (18.8 min, on time) because experience taught it that B is slow in rain. On the 1,000-delivery benchmark training lifted on-time deliveries from 94.0% to 95.0% overall and from 85.3% to 87.6% on urgent orders.

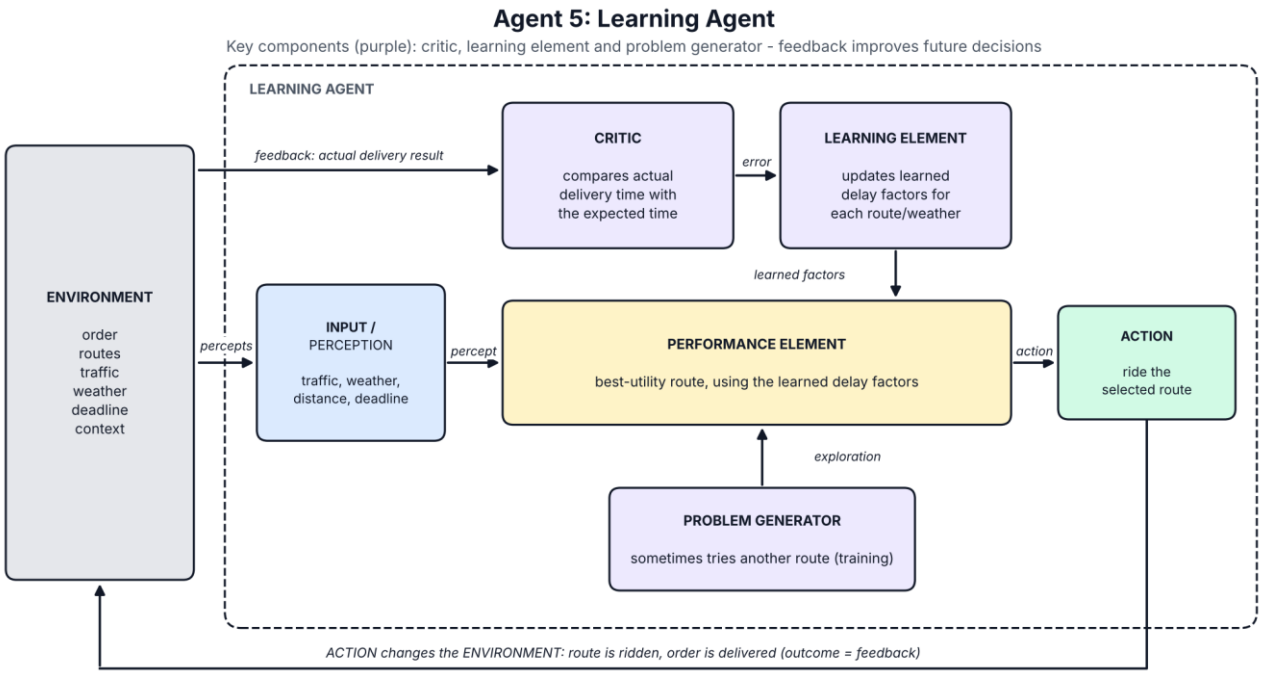


Figure 5. Architecture of Learning Agent.

What it learned (multiplier on the published travel time; the true values are 1.40 for B in rain and 1.30 for A in heavy traffic)

Route	Traffic	Weather	Learned ×	Samples
B	light	rain	1.31	12
B	light	clear	1.05	62
A	heavy	clear	1.36	15
A	heavy	rain	1.29	9
C	normal	rain	1.02	14

The factors are estimates from few samples. Some factors above 1 for situations with no hidden effect (for example A, light, clear: 1.34) come from rush-hour and alert delays that this agent cannot see; they average into its estimate.

Code: `agent5_learning.py`

```
"""Agent 5 - Learning Agent (classic four-part structure).
  Performance element : picks the best-utility route using its LEARNED delay factors
  Critic              : compares what really happened with what it expected
  Learning element    : updates the learned factor for that (route, traffic, weather)
  Problem generator   : while training, sometimes tries another route to gain new experience
Learned knowledge = a multiplier on the published travel time, e.g. 'route B in rain is x1.4'."""
import random
from base import StatefulAgent, Decision, utility
from environment import OVERHEAD_MIN, travel_minutes, route_cost, execute, random_situation


class LearningAgent(StatefulAgent):
    name = "5. Learning"
    EPSILON = 0.25                      # exploration rate (training only)

    def __init__(self, seed=0):
        super().__init__()
        self.factor = {}                # (route, traffic, weather) -> learned multiplier (starts at
1.0)
        self.count = {}
        self.explore = False
        self.rng = random.Random(seed)

    def predicted_eta(self, p, r):
        traffic = self.traffic_of(r)
        f = self.factor.get((r.name, traffic, p.weather), 1.0)
        return OVERHEAD_MIN + travel_minutes(r.distance_km, traffic, p.weather) * f, traffic, f

    def decide(self, p):                # performance element (+ problem generator)
        if self.explore and self.rng.random() < self.EPSILON:
            r = self.rng.choice(p.routes)
            self.commit(r.name)
            return Decision(r.name, "Exploring (problem generator): trying a route to gain
experience")
        scores, factors = {}, {}
        for r in p.routes:
            eta, traffic, f = self.predicted_eta(p, r)
            scores[r.name] = utility(eta, r.distance_km, traffic, route_cost(r.distance_km, r.toll),
p.deadline_min)
            factors[r.name] = f
        best = max(scores, key=scores.get)
        self.commit(best)
        learned = ", ".join(f"{n} x{factors[n]:.2f}" for n in sorted(factors))
        return Decision(best, f"Best utility with learned delay factors ({learned}) -> {best}")

    def learn(self, p, decision, outcome):          # critic + learning element
        r = next(x for x in p.routes if x.name == outcome.route)
        traffic = self.traffic_of(r)
        expected = travel_minutes(r.distance_km, traffic, p.weather)
        observed = (outcome.minutes - OVERHEAD_MIN) / expected          # critic: reality /
expectation
        key = (r.name, traffic, p.weather)
        n = self.count.get(key, 0)
        old = self.factor.get(key, 1.0)
        self.factor[key] = old + (observed - old) / (n + 3)             # learning element: running
average
        self.count[key] = n + 1

    def learned_table(self):
        """What the agent has learned: multiplier on the published travel time per situation."""
        lines = [f"{'Route':<7}{'Traffic':<9}{'Weather':<9}{'Learned x':<11}{'Samples':<8}"]
        for key, f in sorted(self.factor.items()):
            lines.append(f"{key[0]:<7}{key[1]:<9}{key[2]:<9}{f:<11.2f}{self.count[key]:<8}")
        return "\n".join(lines)

    def train(self, episodes=600, seed=1):
        """Gain experience from simulated PAST deliveries (with natural randomness)."""
        rng = random.Random(seed)
        self.explore = True
```

```
        for i in range(episodes):
            s = random_situation(rng, i)
            self.reset()
            self.observe(s.percept("accept"))
            p = s.percept("pickup")
            self.observe(p)
            d = self.decide(p)
            self.learn(p, d, execute(s, d.route, rng))
        self.explore = False
        self.reset()
```

Agent 6 – Hierarchical Agent

Behaviour: the main goal “deliver the order on time” is broken into five subgoals (accept order, choose route, pick up food, deliver order, confirm delivery). A top-level controller runs them in order and each has its own small sub-controller and time budget. The “choose route” sub-controller picks the best-utility route that fits the deadline minus a 10% safety buffer; if none fits it takes the fastest and flags the goal as at risk.

In the tests: route A in Test 1 and route B in Tests 2 and 3, the same as Agent 4. In Test 3 the 10% buffer shrank the budget to 18 min, which ruled out route C (18.2 min predicted) and left B, so the buffer hurt here (B arrived in 21.4 min, late). Its distinctive output is the plan it produces, shown below for Test 3.

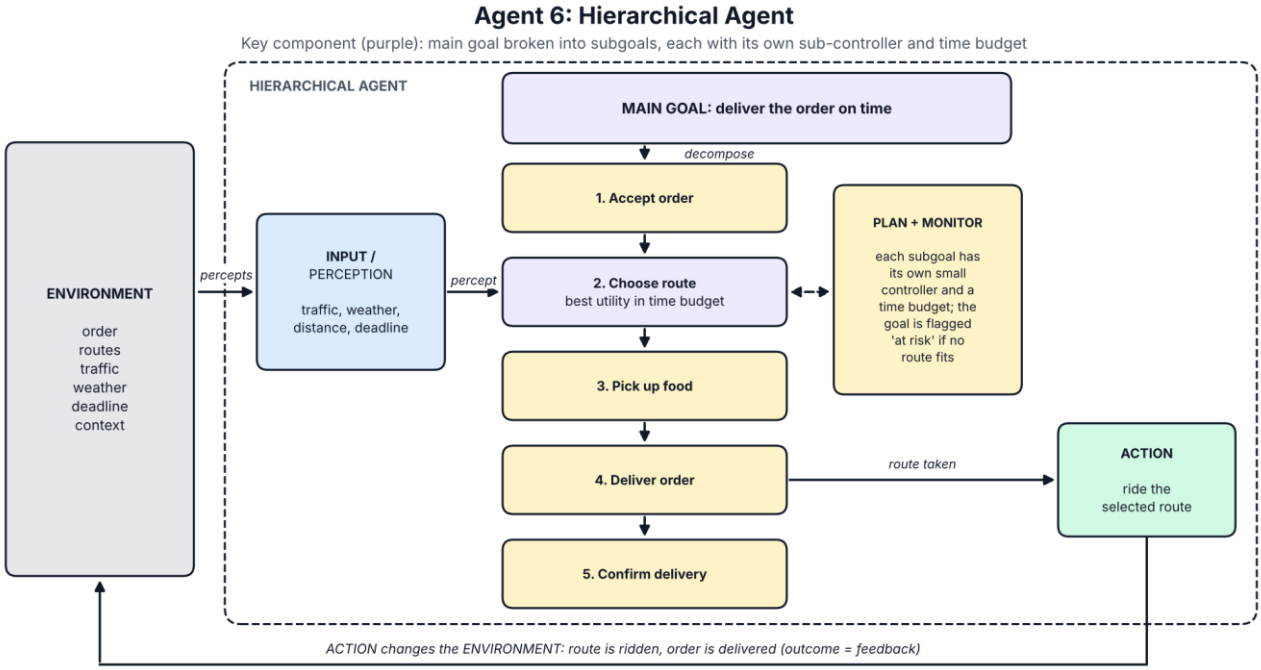


Figure 6. Architecture of Hierarchical Agent.

Plan produced in Test 3

```
1. Accept order: ORD-103 accepted, deadline 20 min (URGENT)
2. Choose route: route B (best utility among routes within the 18-min budget)
3. Pick up food: collect order at restaurant (budget 5 min)
4. Deliver order: ride via B, predicted total 17.3 min
5. Confirm delivery: hand over and confirm (budget 2 min)
```

Code: `agent6_hierarchical.py`

```
"""Agent 6 - Hierarchical Agent.
The main goal 'deliver the order on time' is broken into subgoals. A top-level controller runs the
subgoals in order; each subgoal has its own small sub-controller and its own time budget.
The 'Choose route' sub-controller must respect the time budget left after the other subgoals."""
from base import StatefulAgent, Decision, utility
from environment import nominal_eta, route_cost

SUBGOALS = ["Accept order", "Choose route", "Pick up food", "Deliver order", "Confirm delivery"]
RESERVED_MIN = {"Pick up food": 5, "Confirm delivery": 2}      # time kept for the other subgoals
SAFETY_MARGIN = 0.10                                           # keep 10 % of the deadline as a
buffer


class HierarchicalAgent(StatefulAgent):
    name = "6. Hierarchical"

    # ---- sub-controllers: one per subgoal ------------------------------------------------
    def accept_order(self, p):
        return f"{p.order_id} accepted, deadline {p.deadline_min} min{' (URGENT)' if p.urgent else
''}"

    def choose_route(self, p):
        budget = p.deadline_min * (1 - SAFETY_MARGIN)          # total time we allow ourselves
        options = []
        for r in p.routes:
            traffic = self.traffic_of(r)
            eta = nominal_eta(r.distance_km, traffic, p.weather)
            u = utility(eta, r.distance_km, traffic, route_cost(r.distance_km, r.toll),
p.deadline_min)
            options.append((u, r.name, eta))
        fits = [o for o in options if o[2] <= budget]
        flags = [] if fits else ["goal at risk"]
        u, name, eta = max(fits) if fits else min(options, key=lambda o: o[2])
        how = f"best utility among routes within the {budget:.0f}-min budget" if fits else "nothing
fits the budget: fastest route"
        return name, eta, how, flags

    # ---- top-level controller ------------------------------------------------------------
    def decide(self, p):
        plan, route, eta, flags = [], None, None, []
        for i, goal in enumerate(SUBGOALS, 1):
            if goal == "Accept order":
                plan.append(f"{i}. {goal}: {self.accept_order(p)}")
            elif goal == "Choose route":
                route, eta, how, flags = self.choose_route(p)
                self.commit(route)
                plan.append(f"{i}. {goal}: route {route} ({how})")
            elif goal == "Pick up food":
                plan.append(f"{i}. {goal}: collect order at restaurant (budget {RESERVED_MIN[goal]}
min)")
            elif goal == "Deliver order":
                plan.append(f"{i}. {goal}: ride via {route}, predicted total {eta:.1f} min")
            else:
                plan.append(f"{i}. {goal}: hand over and confirm (budget {RESERVED_MIN[goal]} min)")
        return Decision(route, f"Subgoal 2 chose {route}: {plan[1].split(': ', 1)[1]}", plan, flags)
```

Agent 7 – Contextual Agent

Behaviour: it collects context (time of day, customer type, food type, live road alerts, road attributes plus weather), interprets it, and lets it change two things before the utility choice: the predicted time of each route (rain on an underpass ×1.3, rush hour +10%, alert delays) and the weights (hot food makes time matter more; a VIP customer makes cost matter less and lateness cost twice as much). Its beliefs are close to, but not exactly, the real effects (it expects an accident to cost 10 min, the world charges 12).

In the tests: route A in Test 1 (no context to act on, so it agrees with everyone). Test 2: the accident alert adds 10 min to C (26.0 min), so it took B (15.2 min). Test 3: context-adjusted times were A = 31.1, B = 20.4, C = 19.4 min; the underpass pushed B past the 20-min deadline and hot food plus a VIP customer raised the cost of being late, so it took C and was on time (18.8 min).

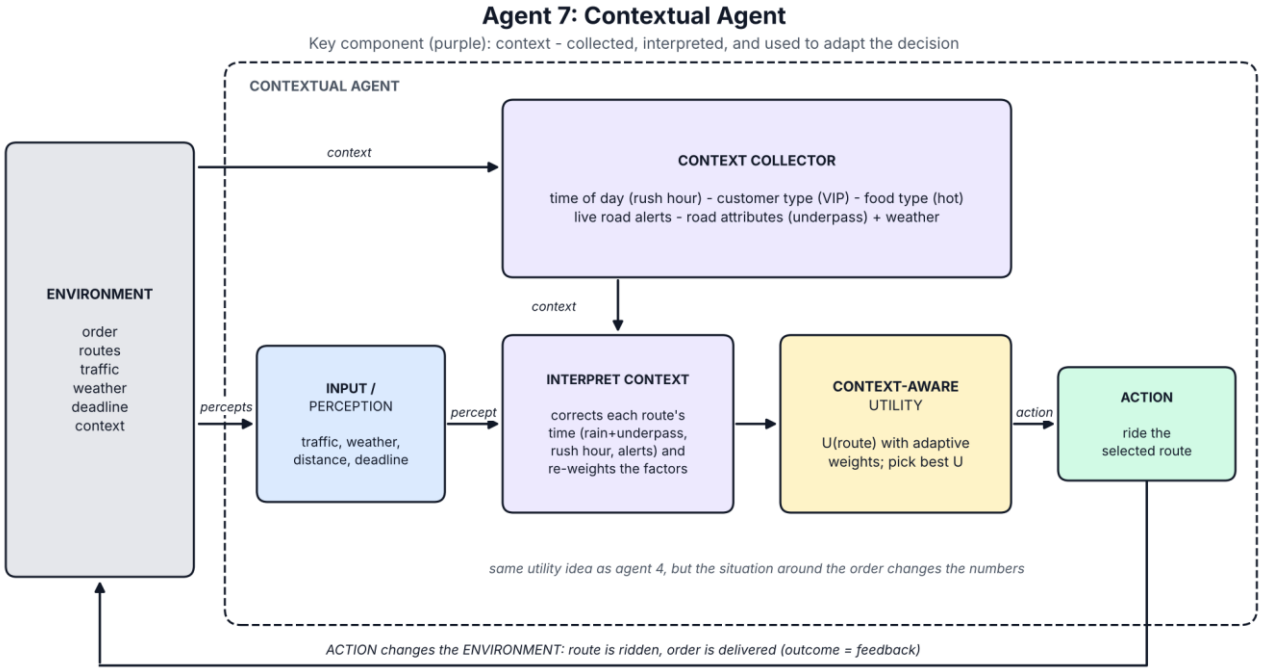


Figure 7. Architecture of Contextual Agent.

Code: `agent7_contextual.py`

```
"""Agent 7 - Contextual Agent.
Collects CONTEXT (time of day, customer type, food type, live road alerts, road attributes +
weather),
interprets it, and lets it change both the predicted delivery time and the importance of each factor
before choosing a route with the utility function."""
from base import StatefulAgent, Decision, utility, DEFAULT_WEIGHTS, LATE_PENALTY
from environment import OVERHEAD_MIN, travel_minutes, route_cost

EXPECTED_ALERT_DELAY = {"accident": 10, "road_works": 6, "procession": 12}   # what the agent
believes an alert costs


class ContextualAgent(StatefulAgent):
    name = "7. Contextual"

    def decide(self, p):
        c = p.context
        # 1) collect + interpret context -> adapt the weights (what matters in THIS delivery)
        w, late_penalty, notes = dict(DEFAULT_WEIGHTS), LATE_PENALTY, []
        if c.food == "hot":
            w["time"] *= 1.5
            notes.append("hot food: time weighs more")
        if c.customer == "vip":
            w["cost"] *= 0.3
            late_penalty *= 2
            notes.append("VIP: cost matters less, lateness penalised x2")
        # 2) use context to correct the predicted travel time of every route
        scores = {}
        for r in p.routes:
            traffic = self.traffic_of(r)
            tt = travel_minutes(r.distance_km, traffic, p.weather)
            if p.weather == "rain" and r.underpass:
                tt *= 1.30
                notes.append(f"rain + underpass on {r.name}: x1.3")
            if c.time_of_day == "rush_hour" and traffic != "light":
                tt *= 1.10
            tt += sum(EXPECTED_ALERT_DELAY[a] for n, a in c.alerts if n == r.name)
            eta = OVERHEAD_MIN + tt
            scores[r.name] = (utility(eta, r.distance_km, traffic, route_cost(r.distance_km,
r.toll),
                                      p.deadline_min, w, late_penalty), eta)
        if c.time_of_day == "rush_hour":
            notes.append("rush hour: +10% on busy routes")
        notes += [f"alert on {n}: {a}" for n, a in c.alerts]
        best = max(scores, key=lambda n: scores[n][0])
        self.commit(best)
        etas = ", ".join(f"{n}={scores[n][1]:.1f}" for n in sorted(scores))
        return Decision(best, f"Context-adjusted ETA ({etas} min) -> {best} | context used: "
                              + ("; ".join(dict.fromkeys(notes)) or "none"))
```

Test results

All seven agents ran on the same three situations (the conditions are in the tables under "The simulated world"). Times are the true delivery times reported by the simulator. The tables below are generated from `results/test_results.csv`, written by `python run_tests.py`; the raw program output is reproduced in Appendix A and each agent's reasoning in Appendix B.

Test 1 – normal traffic, good weather (deadline 35 min)

Agent	Route	Time (min)	On time	Cost (Rs)
1. Simple Reflex	A	14.0	yes	28
2. Model-Based Reflex	A	14.0	yes	28
3. Goal-Based	A	14.0	yes	28
4. Utility-Based	A	14.0	yes	28
5. Learning	A	14.0	yes	28
6. Hierarchical	A	14.0	yes	28
7. Contextual	A	14.0	yes	28

All seven chose route A. Nothing in this situation forces a trade-off, so the architectures agree.

Test 2 – heavy traffic, multiple routes (deadline 30 min; accident alert on C; live feed for B down)

Agent	Route	Time (min)	On time	Cost (Rs)
1. Simple Reflex	C	27.5	yes	96
2. Model-Based Reflex	B	15.2	yes	44
3. Goal-Based	A	29.8	yes	28
4. Utility-Based	B	15.2	yes	44
5. Learning	B	15.2	yes	44
6. Hierarchical	B	15.2	yes	44
7. Contextual	B	15.2	yes	44

Route B (15.2 min, Rs 44) was best. Simple Reflex could not see B because its live feed was down, so it took C, which had an accident that only the Contextual agent knew about (27.5 min, Rs 96 with the toll). Goal-Based took A as soon as its predicted 24.5 min passed the 30-min goal test and arrived 0.2 min inside the deadline.

Test 3 – heavy traffic, rain, urgent (deadline 20 min)

Agent	Route	Time (min)	On time	Cost (Rs)
1. Simple Reflex	B	21.4	NO	44
2. Model-Based Reflex	B	21.4	NO	44
3. Goal-Based	C	18.8	yes	96
4. Utility-Based	B	21.4	NO	44
5. Learning	C	18.8	yes	96
6. Hierarchical	B	21.4	NO	44
7. Contextual	C	18.8	yes	96

Only route C arrives on time. Agents 1, 2, 4 and 6 took B (light traffic, fastest on the feed), but its underpass floods in rain, so they arrived 1.4 min late. Goal-Based reached C through its shortest-first goal test, the Learning agent through experience, and the Contextual agent through context.

Summary: route chosen in each test and deliveries on time

Agent	Test 1	Test 2	Test 3	On time
1. Simple Reflex	A	C	B	2/3
2. Model-Based Reflex	A	B	B	2/3
3. Goal-Based	A	A	C	3/3
4. Utility-Based	A	B	B	2/3
5. Learning	A	B	C	3/3
6. Hierarchical	A	B	B	2/3
7. Contextual	A	B	C	3/3

Three agents were on time in every test, but not for the same reason or by the same margin: Goal-Based got through Test 2 with 0.2 min to spare, while Learning and Contextual chose the fastest route there.

Comparison: how the agents behaved differently

The agents agree when the situation is easy and split as soon as a good decision needs information beyond the current traffic reading.

Agent	Information it uses	How it chooses	What went wrong in the tests
1. Simple Reflex	current percept only	first matching IF-THEN rule	could not see B when its feed was down (Test 2); took B into the flooded underpass (Test 3)
2. Model-Based Reflex	percept + internal state (last known traffic, order status)	IF-THEN rules on the state	trusted the feed: B in rain (Test 3)
3. Goal-Based	state + deadline	first route, shortest first, whose predicted time meets the deadline	took the heavy route A with 0.2 min to spare (Test 2); ignores cost and safety margin
4. Utility-Based	state + deadline + cost	highest weighted utility of time, distance, traffic and cost	trusted the feed: B in rain (Test 3)
5. Learning	as 4, plus past deliveries	utility with learned delay factors	needs many past deliveries; learned factors are approximate
6. Hierarchical	as 4, plus subgoal time budgets	subgoal plan; best utility within the deadline minus 10%	the 10% buffer excluded C in Test 3, leaving B
7. Contextual	as 4, plus context (alerts, rush hour, customer, food, road attributes)	utility with context-corrected times and weights	needs reliable context data; its beliefs are hand-written

Why they differ

- **Memory:** only Agent 1 is memoryless. When B's live feed dropped in Test 2 it lost B; Agents 2 to 7 share a state that remembers the last reading.
- **Decision rule:** rules (1, 2) react to traffic labels; the goal test (3) asks only "does it make the deadline?"; utility (4) trades time against distance, traffic and cost, so it prefers the cheap fast route B over the toll route C.
- **Extra knowledge:** Agents 5 and 7 are the only ones that avoided B's flooded underpass in Test 3. The Learning agent learned it from 600 past deliveries; the Contextual agent inferred it from rain plus an underpass on the route.
- **Structure:** Agent 6 changes how the decision is organised (subgoals and budgets), not what information it uses, so it behaved like Agent 4.

Benchmark: 1,000 unseen random deliveries (307 urgent)

Same road network with random traffic, weather, deadlines and context; the Learning agent was trained on 600 other deliveries. The oracle always rides the truly fastest route and is the upper reference. The table is generated from `results/benchmark.txt` (`python benchmark.py`).

Agent	On time, all (%)	On time, urgent (%)	Average time (min)	Average cost (Rs)
1. Simple Reflex	93.2	83.1	16.5	44
2. Model-Based Reflex	94.4	85.7	16.1	46
3. Goal-Based	88.0	86.6	18.9	33
4. Utility-Based	94.0	85.3	16.4	38
5. Learning (untrained)	94.0	85.3	16.4	38
5. Learning (trained)	95.0	87.6	16.2	40
6. Hierarchical	94.5	87.0	16.3	38
7. Contextual	**96.5**	**92.2**	**15.6**	**42**
Oracle (hindsight)	*96.8*	*92.2*	*15.3*	*53*

- The Contextual agent leads by 1.5 points overall (96.5 against 95.0 for the next best) and by 4.6 points on urgent orders (92.2 against 87.6), where it equals the oracle.

- Goal-Based is the cheapest (Rs 33) but the least often on time overall (88.0%): it takes the shortest route that passes its test and leaves no margin.
- Training lifted the Learning agent from 94.0% to 95.0% overall and from 85.3% to 87.6% on urgent orders.
- Differences of about one point (10 deliveries in 1,000) are small, so the order of Agents 2, 4, 5 and 6 should not be over-read; the Contextual lead and the weak urgent score of Agent 1 (83.1%) are the clearer signals.

Conclusion

The Contextual Agent was the most suitable for this scenario. It was on time in all three tests, chose the fastest route in Test 2, and came closest to the hindsight oracle on 1,000 unseen deliveries: 96.5% on time against 96.8%, and 92.2% on urgent orders, equal to the oracle, with the lowest average delivery time of the seven agents (15.6 min).

Why it fits:

- **Delivery is a trade-off.** Time, distance, traffic and cost pull in different directions, which a utility score handles better than IF-THEN rules or a yes/no goal test. Agents 4 to 7 all reason this way.
- **What decides a delivery is context.** Rain on an underpass, an accident, rush hour, a VIP customer or hot food are not traffic readings. The Contextual agent acts on them the moment they appear, while the Learning agent needed 600 past deliveries to learn just one such effect.
- **The other agents each stop short.** Simple Reflex and Model-Based Reflex ignore the deadline and cost; Goal-Based accepts the first route that scrapes through; Utility trusts the traffic feed; Hierarchical organises the work but does not know more; Learning improves only as fast as it gathers experience.

Its weakness is that it depends on reliable context data and hand-written beliefs. In a real system the best design would combine them: a contextual utility agent for the choice, the hierarchical structure to run the order from acceptance to confirmation, and a learning component that corrects the contextual beliefs from delivery feedback. All of this is shown in a simulator built for this assignment, so it illustrates how the architectures use information rather than proving real-world performance.

Shared code and how to run it

File	Purpose
`environment.py`	world model, ground-truth simulator, the three test situations, random situation generator
`base.py`	agent interface, internal-state helper, utility function
`agent1_simple_reflex.py` … `agent7_contextual.py`	the seven agents (listed above)
`run_tests.py`	runs the three tests, prints the tables, saves them to `results/`
`benchmark.py`	the 1,000-delivery benchmark
`diagram_lib.py`, `diagrams_1_4.py`, `diagrams_5_7.py`	draw the seven architecture diagrams into `diagrams/`

```
cd food_delivery_agents
python run_tests.py          # the three test cases (add --why to see each agent's reasoning)
python benchmark.py          # the 1,000-delivery benchmark
python diagrams_5_7.py       # redraw the seven diagrams (needs matplotlib)
```

The agents, tests and benchmark use only the Python standard library (tested on Python 3.13); matplotlib is needed only for the diagrams. Every random choice is seeded, so the results are identical on every run.

Code: `environment.py`

```
"""environment.py - the shared world used by all seven agents.

Two views of the world are kept apart on purpose:
  * Situation = the TRUE world (only the simulator uses it, to judge a decision afterwards)
  * Percept   = what an agent is allowed to SENSE (a live traffic reading can be missing)
"""
from __future__ import annotations
from dataclasses import dataclass

# ---- nominal model: what the live traffic feed lets an agent predict -------------------------
SPEED_KMH = {"light": 40, "normal": 30, "heavy": 12}    # km/h for each traffic level
RAIN_FACTOR = 0.8                                        # rain slows every route by 20 %
OVERHEAD_MIN = 7                                         # pick-up wait (5) + hand-over/confirm (2)
FUEL_RS_PER_KM = 8
TRAFFIC_SCORE = {"light": 0.0, "normal": 0.5, "heavy": 1.0}
TRAFFIC_RANK = {"light": 0, "normal": 1, "heavy": 2}


def travel_minutes(distance_km, traffic, weather):
    speed = SPEED_KMH[traffic] * (RAIN_FACTOR if weather == "rain" else 1.0)
    return distance_km / speed * 60


def nominal_eta(distance_km, traffic, weather):
    return OVERHEAD_MIN + travel_minutes(distance_km, traffic, weather)


def route_cost(distance_km, toll):
    return distance_km * FUEL_RS_PER_KM + toll


# ---- what an agent can perceive ---------------------------------------------------------------
@dataclass(frozen=True)
class Context:
    time_of_day: str = "afternoon"     # "rush_hour" | "afternoon" | "night"
    customer: str = "regular"          # "regular" | "vip"
    food: str = "standard"             # "standard" | "hot"
    alerts: tuple = ()                 # live road alerts, e.g. (("C", "accident"),)


@dataclass(frozen=True)
class Route:                           # TRUE state of a route
    name: str
    distance_km: float
```

```
    traffic: str                        # "light" | "normal" | "heavy"
    toll: int = 0                       # Rs
    underpass: bool = False             # static road attribute (floods when it rains)

@dataclass(frozen=True)
class RouteView:                        # what an agent senses; traffic is None if the feed is down
    name: str
    distance_km: float
    traffic: str | None
    toll: int
    underpass: bool

@dataclass(frozen=True)
class Percept:
    stage: str                          # "accept" (order arrives) or "pickup" (decision time)
    order_id: str
    urgent: bool
    deadline_min: int
    weather: str                        # "clear" | "rain"
    routes: tuple
    context: Context

@dataclass(frozen=True)
class Situation:                        # the TRUE world for one delivery
    name: str
    order_id: str
    weather: str
    urgent: bool
    deadline_min: int
    routes: tuple
    context: Context
    feed_down_at_pickup: tuple = ()     # routes whose live traffic feed is down at decision time

    def percept(self, stage):
        down = self.feed_down_at_pickup if stage == "pickup" else ()
        views = tuple(RouteView(r.name, r.distance_km, None if r.name in down else r.traffic,
                                r.toll, r.underpass) for r in self.routes)
        return Percept(stage, self.order_id, self.urgent, self.deadline_min,
                       self.weather, views, self.context)

# ---- ground truth: the simulator that judges every decision -----------------------------------
ALERT_DELAY_MIN = {"accident": 12, "road_works": 8, "procession": 15}   # true extra minutes

def hidden_factor(route, weather):
    """Real effects the live traffic feed does NOT show. Agents can only discover them from
    history (learning agent) or infer them from road attributes (contextual agent)."""
    f = 1.0
    if route.name == "A" and route.traffic == "heavy":
        f *= 1.30                       # market spill-over: jams are worse than reported
    if route.underpass and weather == "rain":
        f *= 1.40                       # underpass waterlogs in rain
    if route.toll:
        f *= 0.95                       # toll road is slightly faster than reported
    return f

@dataclass(frozen=True)
class Outcome:
    route: str
    minutes: float                      # true total delivery time
    on_time: bool
    late_by: float
    cost: float

def execute(s: Situation, route_name, rng=None) -> Outcome:
    """Carry out the delivery on the chosen route and report what really happened."""
    r = next(x for x in s.routes if x.name == route_name)
```

```
    t = travel_minutes(r.distance_km, r.traffic, s.weather) * hidden_factor(r, s.weather)
    if s.context.time_of_day == "rush_hour" and r.traffic != "light":
        t *= 1.10
    t += sum(ALERT_DELAY_MIN[a] for n, a in s.context.alerts if n == route_name)
    if rng is not None:                # noise is used for past deliveries, not for the 3 tests
        t *= rng.gauss(1.0, 0.05)
    total = OVERHEAD_MIN + t
    return Outcome(route_name, total, total <= s.deadline_min,
                   max(0.0, total - s.deadline_min), route_cost(r.distance_km, r.toll))

# ---- the three required test situations (same road network, different conditions) ------------
def _net(a, b, c):
    return (Route("A", 3.5, a), Route("B", 5.5, b, 0, True), Route("C", 4.5, c, 60))

def test_situations():
    return [
        Situation("Test 1 - normal traffic, good weather", "ORD-101", "clear", False, 35,
                  _net("normal", "normal", "normal"), Context()),
        Situation("Test 2 - heavy traffic, multiple routes", "ORD-102", "clear", False, 30,
                  _net("heavy", "light", "normal"), Context(alerts=(("C", "accident"),)),
                  feed_down_at_pickup=("B",)),
        Situation("Test 3 - heavy traffic, rain, urgent", "ORD-103", "rain", True, 20,
                  _net("heavy", "light", "normal"), Context("rush_hour", "vip", "hot")),
    ]

# ---- random situations: past deliveries for training and a held-out benchmark ----------------
ROAD_NETWORK = (("A", 3.5, 0, False), ("B", 5.5, 0, True), ("C", 4.5, 60, False))

def random_situation(rng, i=0) -> Situation:
    urgent = rng.random() < 0.3
    routes = tuple(Route(n, km, rng.choices(["light", "normal", "heavy"], [0.3, 0.4, 0.3])[0], toll,
up)
                   for n, km, toll, up in ROAD_NETWORK)
    ctx = Context(rng.choice(["rush_hour", "afternoon", "night"]),
                  "vip" if rng.random() < 0.15 else "regular",
                  "hot" if rng.random() < 0.4 else "standard",
                  ((rng.choice("ABC"), rng.choice(list(ALERT_DELAY_MIN))),) if rng.random() < 0.25
else ())
    return Situation(f"random-{i}", f"ORD-{i}", "rain" if rng.random() < 0.3 else "clear", urgent,
                     rng.randint(18, 24) if urgent else rng.randint(28, 40), routes, ctx,
                     (rng.choice("ABC"),) if rng.random() < 0.2 else ())
```

Code: `base.py`

```
"""base.py - common agent interface, internal-state helper and the shared utility function."""
from __future__ import annotations
from dataclasses import dataclass, field
from environment import TRAFFIC_SCORE

DEFAULT_WEIGHTS = {"time": 0.50, "distance": 0.15, "traffic": 0.15, "cost": 0.20}
LATE_PENALTY = 3.0


@dataclass
class Decision:
    route: str                                  # the action: which route to take
    reason: str                                 # one-line explanation of why
    plan: list = field(default_factory=list)    # ordered subtasks (hierarchical agent)
    flags: list = field(default_factory=list)   # warnings, e.g. "goal at risk"


class Agent:
    name = "Agent"

    def reset(self):                            # clear short-term memory before each delivery
        pass

    def observe(self, percept):                 # perception update
        pass
```

```
    def decide(self, percept) -> Decision:       # choose the action
        raise NotImplementedError

    def learn(self, percept, decision, outcome):  # feedback after the delivery
        pass


class StatefulAgent(Agent):
    """Keeps an internal state: order status, current route, weather and the LAST KNOWN traffic
    of every route (so a missing live reading does not blind the agent)."""

    def __init__(self):
        self.reset()

    def reset(self):
        self.state = {"order_status": "new", "current_route": None, "weather": None, "traffic": {}}

    def observe(self, p):
        st = self.state
        st["weather"] = p.weather
        for r in p.routes:
            if r.traffic is not None:
                st["traffic"][r.name] = r.traffic
        st["order_status"] = {"accept": "accepted", "pickup": "picked_up"}.get(p.stage,
st["order_status"])

    def traffic_of(self, r):
        """Live reading if available, otherwise the remembered one, otherwise assume 'normal'."""
        return r.traffic or self.state["traffic"].get(r.name, "normal")

    def commit(self, route):
        self.state["current_route"] = route
        self.state["order_status"] = "en_route"


def utility(eta, distance_km, traffic, cost, deadline, w=DEFAULT_WEIGHTS,
late_penalty=LATE_PENALTY):
    """Higher is better. Every factor is scaled to about 0..1, weighted, and subtracted;
    arriving after the deadline adds a large extra penalty that grows with the lateness."""
    loss = (w["time"] * eta / deadline + w["distance"] * distance_km / 10
            + w["traffic"] * TRAFFIC_SCORE[traffic] + w["cost"] * cost / 100)
    if eta > deadline:
        loss += late_penalty * (1 + (eta - deadline) / deadline)
    return -loss
```

Code: `run_tests.py`

```
"""run_tests.py - runs all seven agents on the three required delivery situations.
Usage:  python run_tests.py          (compact tables)
        python run_tests.py --why    (also print each agent's reasoning and the hierarchical plan)
Files are written to results/ (test_results.txt, test_results.csv, decision_log.txt,
learned_factors.txt)."""
import csv
import os
import sys

from environment import test_situations, execute
from agent1_simple_reflex import SimpleReflexAgent
from agent2_model_based import ModelBasedReflexAgent
from agent3_goal_based import GoalBasedAgent
from agent4_utility_based import UtilityBasedAgent
from agent5_learning import LearningAgent
from agent6_hierarchical import HierarchicalAgent
from agent7_contextual import ContextualAgent

OUT = os.path.join(os.path.dirname(os.path.abspath(__file__)), "results")


def build_agents(train=True):
    learner = LearningAgent()
    if train:
        learner.train()                      # experience from 600 simulated past deliveries
    return [SimpleReflexAgent(), ModelBasedReflexAgent(), GoalBasedAgent(), UtilityBasedAgent(),
```

```
            learner, HierarchicalAgent(), ContextualAgent()]

def run_one(agent, s):
    """Perceive -> decide -> act in the environment. Returns (decision, true outcome)."""
    agent.reset()
    agent.observe(s.percept("accept"))       # order arrives
    p = s.percept("pickup")                  # decision time (a traffic feed may be down)
    agent.observe(p)
    d = agent.decide(p)
    return d, execute(s, d.route)


def conditions(s):
    traffic = " ".join(f"{r.name}={r.traffic}" for r in s.routes)
    extra = []
    if s.context.alerts:
        extra.append("alerts: " + ", ".join(f"{n} {a}" for n, a in s.context.alerts))
    if s.feed_down_at_pickup:
        extra.append("live feed down: " + ",".join(s.feed_down_at_pickup))
    c = s.context
    return (f"weather={s.weather} | urgent={'yes' if s.urgent else 'no'} | deadline={s.deadline_min}
min | "
            f"traffic {traffic} | {c.time_of_day}, {c.customer}, {c.food} food" + "".join(" | " + e
for e in extra))


def fmt(route, o):
    return f"{route} {o.minutes:.1f} min ({'on time' if o.on_time else 'LATE'})"


def main():
    why = "--why" in sys.argv
    os.makedirs(OUT, exist_ok=True)
    agents = build_agents()
    table, log, rows, summary, trained = [], [], [], {a.name: [] for a in agents}, {}
    for s in test_situations():
        table += [f"\n=== {s.name} ===", conditions(s),
                  f"{'Agent':<24}{'Route':<7}{'Time(min)':<11}{'On time':<9}{'Cost(Rs)':<9}"]
        log.append(f"\n=== {s.name} ===")
        for a in agents:
            d, o = run_one(a, s)
            table.append(f"{a.name:<24}{d.route:<7}{o.minutes:<11.1f}{'yes' if o.on_time else
'NO':<9}{o.cost:<9.0f}")
            log.append(f"[{a.name}] -> {d.route}: {d.reason}" + (f"  FLAGS: {d.flags}" if d.flags
else ""))
            log += [f"      {step}" for step in d.plan]
            rows.append([s.name, a.name, d.route, round(o.minutes, 1), o.on_time, round(o.cost)])
            summary[a.name].append((d.route, o.on_time))
            if isinstance(a, LearningAgent):
                trained[s.name] = (d.route, o)
    table += ["\n=== Summary: route chosen in Test 1 / 2 / 3 and deliveries on time ===",
              f"{'Agent':<24}{'T1':<5}{'T2':<5}{'T3':<5}{'On time':<8}"]
    for name, res in summary.items():
        table.append(f"{name:<24}" + "".join(f"{r:<5}" for r, _ in res) + f"{sum(ok for _, ok in
res)}/3")
    table.append("\n=== Learning agent: before vs after training on 600 past deliveries ===")
    before = LearningAgent()
    for s in test_situations():
        d0, o0 = run_one(before, s)
        table.append(f"{s.name[:6]}: before -> {fmt(d0.route, o0)} | after ->
{fmt(*trained[s.name])}")
    text = "\n".join(table)
    print(text)
    if why:
        print("\n".join(log))
    learner = next(a for a in agents if isinstance(a, LearningAgent))
    for fname, content in [("test_results.txt", text), ("decision_log.txt", "\n".join(log)),
                           ("learned_factors.txt", learner.learned_table())]:
        with open(os.path.join(OUT, fname), "w") as f:
            f.write(content + "\n")
```

```
    with open(os.path.join(OUT, "test_results.csv"), "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["test", "agent", "route", "minutes", "on_time", "cost_rs"])
        w.writerows(rows)

if __name__ == "__main__":
    main()
```

Code: `benchmark.py`

```
"""benchmark.py - robustness check beyond the 3 hand-made tests: 1000 unseen random deliveries.
'Urgent' = the tight-deadline subset, where choosing the right route matters most.
The learning agent is shown twice: before training and after training on 600 past deliveries."""
import os
import random

from environment import random_situation, execute
from run_tests import build_agents, run_one, OUT
from agent5_learning import LearningAgent

N = 1000
SEED = 2026                                  # different from the training seed, so situations are
unseen


def evaluate(decide_route, situations):
    res = [(s.urgent, execute(s, decide_route(s))) for s in situations]
    everything, urgent = [o for _, o in res], [o for u, o in res if u]
    pct = lambda xs: 100 * sum(o.on_time for o in xs) / len(xs)
    return (pct(everything), pct(urgent), sum(o.minutes for o in everything) / len(everything),
            sum(o.cost for o in everything) / len(everything))


def main():
    rng = random.Random(SEED)
    situations = [random_situation(rng, i) for i in range(N)]
    n_urgent = sum(s.urgent for s in situations)
    rows = []
    untrained = LearningAgent()
    untrained.name = "5. Learning (untrained)"
    for a in [untrained] + build_agents():
        rows.append((a.name, evaluate(lambda s, a=a: run_one(a, s)[0].route, situations)))
    # hindsight reference: always take the truly fastest route (knows every hidden effect)
    rows.append(("Oracle (hindsight)", evaluate(
        lambda s: min(s.routes, key=lambda r: execute(s, r.name).minutes).name, situations)))
    lines = [f"Benchmark on {N} unseen random deliveries ({n_urgent} urgent)",
             f"{'Agent':<26}{'On time % (all)':<17}{'On time % (urgent)':<20}{'Avg min':<9}{'Avg
cost Rs':<11}"]
    lines += [f"{n:<26}{r[0]:<17.1f}{r[1]:<20.1f}{r[2]:<9.1f}{r[3]:<11.0f}" for n, r in rows]
    text = "\n".join(lines)
    print(text)
    with open(os.path.join(OUT, "benchmark.txt"), "w") as f:
        f.write(text + "\n")


if __name__ == "__main__":
    main()
```

Appendix A: Program output

Verbatim contents of `results/test_results.txt`, produced by `python run_tests.py`.

```
=== Test 1 - normal traffic, good weather ===
weather=clear | urgent=no | deadline=35 min | traffic A=normal B=normal C=normal | afternoon,
regular, standard food
Agent                   Route  Time(min)  On time  Cost(Rs)
1. Simple Reflex        A      14.0       yes      28
2. Model-Based Reflex   A      14.0       yes      28
3. Goal-Based           A      14.0       yes      28
4. Utility-Based        A      14.0       yes      28
5. Learning             A      14.0       yes      28
6. Hierarchical         A      14.0       yes      28
7. Contextual           A      14.0       yes      28

=== Test 2 - heavy traffic, multiple routes ===
weather=clear | urgent=no | deadline=30 min | traffic A=heavy B=light C=normal | afternoon, regular,
standard food | alerts: C accident | live feed down: B
Agent                   Route  Time(min)  On time  Cost(Rs)
1. Simple Reflex        C      27.5       yes      96
2. Model-Based Reflex   B      15.2       yes      44
3. Goal-Based           A      29.8       yes      28
4. Utility-Based        B      15.2       yes      44
5. Learning             B      15.2       yes      44
6. Hierarchical         B      15.2       yes      44
7. Contextual           B      15.2       yes      44

=== Test 3 - heavy traffic, rain, urgent ===
weather=rain | urgent=yes | deadline=20 min | traffic A=heavy B=light C=normal | rush_hour, vip, hot
food
Agent                   Route  Time(min)  On time  Cost(Rs)
1. Simple Reflex        B      21.4       NO       44
2. Model-Based Reflex   B      21.4       NO       44
3. Goal-Based           C      18.8       yes      96
4. Utility-Based        B      21.4       NO       44
5. Learning             C      18.8       yes      96
6. Hierarchical         B      21.4       NO       44
7. Contextual           C      18.8       yes      96

=== Summary: route chosen in Test 1 / 2 / 3 and deliveries on time ===
Agent                   T1   T2   T3   On time
1. Simple Reflex        A    C    B    2/3
2. Model-Based Reflex   A    B    B    2/3
3. Goal-Based           A    A    C    3/3
4. Utility-Based        A    B    B    2/3
5. Learning             A    B    C    3/3
6. Hierarchical         A    B    B    2/3
7. Contextual           A    B    C    3/3

=== Learning agent: before vs after training on 600 past deliveries ===
Test 1: before -> A 14.0 min (on time) | after -> A 14.0 min (on time)
Test 2: before -> B 15.2 min (on time) | after -> B 15.2 min (on time)
Test 3: before -> B 21.4 min (LATE) | after -> C 18.8 min (on time)
```

Appendix B: Decision log

Verbatim contents of `results/decision_log.txt` (`python run_tests.py --why`): the route each agent chose in each test and the reason it gave. Agent 6 also prints its five-step plan.

```
=== Test 1 - normal traffic, good weather ===
[1. Simple Reflex] -> A: R3: OTHERWISE take the shortest route
[2. Model-Based Reflex] -> A: R3: otherwise shortest route | state: status=en_route, weather=clear,
traffic={'A': 'normal', 'B': 'normal', 'C': 'normal'}
[3. Goal-Based] -> A: Goal test passed: predicted 14.0 min <= 35 min (checked A=14.0)
[4. Utility-Based] -> A: Highest utility -> A | A: U=-0.38 (14 min, Rs 28), B: U=-0.50 (18 min, Rs
44), C: U=-0.56 (16 min, Rs 96)
[5. Learning] -> A: Best utility with learned delay factors (A x1.14, B x1.08, C x1.18) -> A
[6. Hierarchical] -> A: Subgoal 2 chose A: route A (best utility among routes within the 32-min
budget)
      1. Accept order: ORD-101 accepted, deadline 35 min
      2. Choose route: route A (best utility among routes within the 32-min budget)
      3. Pick up food: collect order at restaurant (budget 5 min)
      4. Deliver order: ride via A, predicted total 14.0 min
      5. Confirm delivery: hand over and confirm (budget 2 min)
[7. Contextual] -> A: Context-adjusted ETA (A=14.0, B=18.0, C=16.0 min) -> A | context used: none

=== Test 2 - heavy traffic, multiple routes ===
[1. Simple Reflex] -> C: R1: IF shortest route is HEAVY THEN take the lightest-traffic alternative
[2. Model-Based Reflex] -> B: R1: state says A is heavy -> lightest alternative (used REMEMBERED
traffic for B: live feed down) | state: status=en_route, weather=clear, traffic={'A': 'heavy', 'B':
'light', 'C': 'normal'}
[3. Goal-Based] -> A: Goal test passed: predicted 24.5 min <= 30 min (checked A=24.5)
[4. Utility-Based] -> B: Highest utility -> B | A: U=-0.67 (24 min, Rs 28), B: U=-0.42 (15 min, Rs
44), C: U=-0.60 (16 min, Rs 96)
[5. Learning] -> B: Best utility with learned delay factors (A x1.36, B x1.05, C x1.18) -> B
[6. Hierarchical] -> B: Subgoal 2 chose B: route B (best utility among routes within the 27-min
budget)
      1. Accept order: ORD-102 accepted, deadline 30 min
      2. Choose route: route B (best utility among routes within the 27-min budget)
      3. Pick up food: collect order at restaurant (budget 5 min)
      4. Deliver order: ride via B, predicted total 15.2 min
      5. Confirm delivery: hand over and confirm (budget 2 min)
[7. Contextual] -> B: Context-adjusted ETA (A=24.5, B=15.2, C=26.0 min) -> B | context used: alert
on C: accident

=== Test 3 - heavy traffic, rain, urgent ===
[1. Simple Reflex] -> B: R1: IF shortest route is HEAVY THEN take the lightest-traffic alternative
[2. Model-Based Reflex] -> B: R1: state says A is heavy -> lightest alternative | state:
status=en_route, weather=rain, traffic={'A': 'heavy', 'B': 'light', 'C': 'normal'}
[3. Goal-Based] -> C: Goal test passed: predicted 18.2 min <= 20 min (checked A=28.9, C=18.2)
[4. Utility-Based] -> B: Highest utility -> B | A: U=-5.31 (29 min, Rs 28), B: U=-0.60 (17 min, Rs
44), C: U=-0.79 (18 min, Rs 96)
[5. Learning] -> C: Best utility with learned delay factors (A x1.29, B x1.31, C x1.02) -> C
[6. Hierarchical] -> B: Subgoal 2 chose B: route B (best utility among routes within the 18-min
budget)
      1. Accept order: ORD-103 accepted, deadline 20 min (URGENT)
      2. Choose route: route B (best utility among routes within the 18-min budget)
      3. Pick up food: collect order at restaurant (budget 5 min)
      4. Deliver order: ride via B, predicted total 17.3 min
      5. Confirm delivery: hand over and confirm (budget 2 min)
[7. Contextual] -> C: Context-adjusted ETA (A=31.1, B=20.4, C=19.4 min) -> C | context used: hot
food: time weighs more; VIP: cost matters less, lateness penalised x2; rain + underpass on B: x1.3;
rush hour: +10% on busy routes
```

Appendix C: Benchmark and learned factors

Verbatim contents of `results/benchmark.txt` and `results/learned_factors.txt` (what the Learning agent knows after training on 600 past deliveries).

```
Benchmark on 1000 unseen random deliveries (307 urgent)
Agent                      On time % (all)  On time % (urgent)  Avg min  Avg cost Rs
5. Learning (untrained)    94.0             85.3                16.4     38
1. Simple Reflex           93.2             83.1                16.5     44
2. Model-Based Reflex      94.4             85.7                16.1     46
3. Goal-Based              88.0             86.6                18.9     33
4. Utility-Based           94.0             85.3                16.4     38
5. Learning                95.0             87.6                16.2     40
6. Hierarchical            94.5             87.0                16.3     38
7. Contextual              96.5             92.2                15.6     42
Oracle (hindsight)         96.8             92.2                15.3     53

Route  Traffic  Weather  Learned x  Samples
A      heavy    clear    1.36       15
A      heavy    rain     1.29       9
A      light    clear    1.34       95
A      light    rain     1.19       55
A      normal   clear    1.14       135
A      normal   rain     1.15       43
B      heavy    clear    1.10       11
B      heavy    rain     1.25       3
B      light    clear    1.05       62
B      light    rain     1.31       12
B      normal   clear    1.08       45
B      normal   rain     1.37       9
C      heavy    clear    1.02       8
C      heavy    rain     1.01       3
C      light    clear    1.11       30
C      light    rain     1.00       22
C      normal   clear    1.18       29
C      normal   rain     1.02       14
```